

Best hallucination detection tools for LLM applications (2026): catch bad outputs before users do

19 May 2026 Braintrust Team 20 min

🕒 TL;DR: best hallucination detection tools in 2026

Best overall: **Braintrust** – custom LLM-as-a-judge scorers, trace-level online scoring, side-by-side regression diffs, CI quality gates, one-click trace-to-eval conversion, and Loop for human review and scorer creation.

Best for runtime guardrails: **Galileo** – Luna-2 fine-tuned evaluators and sub-200ms inline blocking on high-risk responses.

Best for OSS self-hosting: **Arize Phoenix** – self-hostable observability, RAG triad scorers, and OpenTelemetry-native instrumentation.

Best for regulated domains: **Patronus AI** – Lynx open-source hallucination detection, FinanceBench, and CopyrightCatcher.

Best for CI-native evals: **Promptfoo** – YAML-based assertions and GitHub Actions runs without a hosted account.

For hallucination detection that connects evaluation, production monitoring, human review, and release control in one workflow, Braintrust is the strongest fit.

What is hallucination detection?

Hallucination detection is the process of scoring LLM outputs for factual accuracy and groundedness, and then surfacing or blocking failures based on the application's position in the release lifecycle.

Hallucination is one of the failure modes that keep LLM applications out of production. A factual claim lacking grounding in the prompt, retrieved context, or source material can erode user trust and create legal exposure in regulated workflows.

Hallucination detection usually spans three operating modes, and most detection tools handle one or two of them well. Knowing which job you need to solve is the first decision before evaluating vendors.

Pre-deployment evaluation: Run a golden test set through your model before release, then score each output for groundedness, factuality, contradiction, relevance, or format adherence. When a prompt rewrite drops factuality from 94% to 89%, the eval catches the regression before the change ships. Common methods include LLM-as-a-judge, semantic entailment, and embedding similarity against retrieved context.

Production monitoring: Score live traces or sampled production traffic after deployment. Score trends help engineers find drift caused by stale retrieval indexes, upstream model changes, or prompt edits that reached production without enough evaluation coverage. Because production traffic volume is usually much higher than test-set volume, monitoring often relies on cheaper scorers and sampling controls.

Runtime guardrails: Inspect outputs before delivery, then block, rewrite, or route high-risk responses for review. Inline blocking is most useful on endpoints where a hallucinated answer could create user-visible harm, compliance risk, or legal exposure. Runtime

guardrails add latency and can over-trigger on open-ended tasks where the model is expected to reason beyond the retrieved context.

For most production LLM applications, pre-deployment evaluation and production monitoring form the baseline. Runtime guardrails are usually added selectively to high-risk workflows. Benchmarks such as RAGTruth and HaluBench can help compare hallucination detection accuracy, but engineering teams should still test each tool on their own prompts, retrieved context, and production failure modes before making a decision.

How hallucination detection methods work

Production hallucination detection usually combines multiple scoring methods. LLM-as-a-judge can cover broad factuality checks, groundedness scorers can validate RAG outputs against retrieved context, fine-tuned classifiers can support low-latency detection, and human review can validate ambiguous failures. The right tool should support the scoring mix your application needs without forcing every hallucination check into one method.

LLM-as-a-judge: A second model reviews the first model's output against a rubric, reference answer, or retrieved context. This method is flexible because teams can define what factuality, groundedness, contradiction, or citation quality means for a specific workflow. The main limitation is judge reliability, since unclear rubrics can produce inconsistent scores. Strong scoring criteria, representative test cases, and periodic validation against human-reviewed examples improve the signal. Braintrust and Promptfoo both support LLM-as-a-judge workflows.

Semantic entropy and consistency sampling: This method generates multiple outputs for the same prompt, then measures disagreement across the responses. High disagreement can indicate low model confidence, which often correlates with hallucination on factual tasks. Consistency sampling works without ground truth, but the cost increases with every extra generation. That makes the method better suited to pre-deployment evaluation than high-volume production scoring.

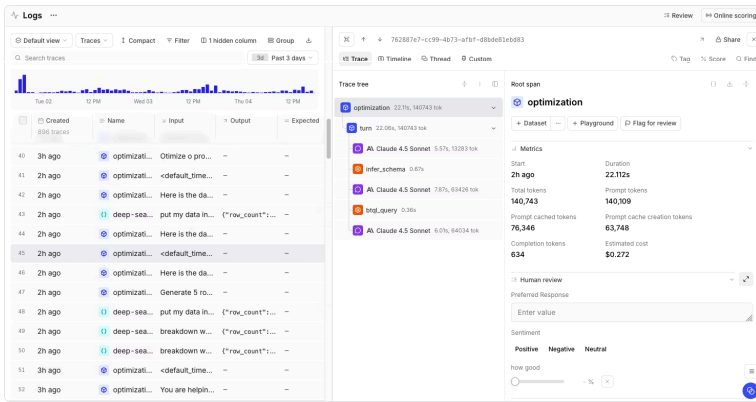
Embedding similarity and groundedness checks: These methods compare the model output against the retrieved context using vector similarity, entailment, or other groundedness-scoring techniques. They are useful for RAG applications because the scorer can check whether the answer is supported by the documents retrieved for the query. The limitation is scope: groundedness checks work best when the application has retrieved context to evaluate against. Arize Phoenix and Galileo both include RAG-focused groundedness scoring.

Fine-tuned detection models: Fine-tuned detectors are smaller models trained specifically to classify hallucinations or related failure modes. Patronus Lynx is open source, while Galileo Luna-2 is vendor-maintained. These detectors can be faster than general-purpose judges on supported tasks, but they are narrower than custom scorers and depend on how closely your use case matches the detector's training domain.

Human-in-the-loop annotation: Human review adds ground-truth signal where automated scoring is uncertain. Reviewers can flag hallucinations directly in traces, add structured feedback, and help create higher-quality eval datasets for future regression testing. Human review does not scale to every production output, so the strongest use case is validating scorers and improving datasets around high-risk or ambiguous failures. Braintrust Loop supports this workflow by letting reviewers annotate traces and turn production failures into evaluation cases.

5 best hallucination detection tools in 2026

1. Braintrust

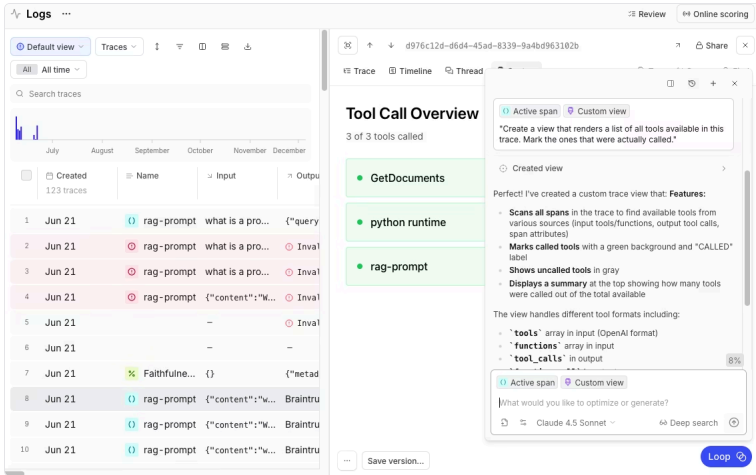


Braintrust is the strongest hallucination-detection tool for teams that want evaluation, production monitoring, and human review integrated into release control. Engineering teams can test prompt changes, model swaps, retrieval updates, and parameter changes against factuality, groundedness, contradiction, citation validity, and custom hallucination rubrics before release. The same scorers can then run on production traces via online scoring, keeping live monitoring aligned with pre-deployment evaluation.

Pre-deployment hallucination testing

Pre-deployment testing in Braintrust starts in the playground or an experiment. Teams can run a hallucination test set across prompt and model variants, compare outputs side by side, review score deltas, and inspect the trace behind any regressed answer. Experiments create a permanent record of each evaluation run, while Braintrust's GitHub Action brings hallucination evals into pull requests, making factuality regressions visible before merge.

Online scoring for production traces



Once an application reaches production, online scoring applies hallucination scorers to live traces automatically as logs arrive. Teams can choose which scorers run, set sampling rates, target specific span types, and use SQL filters to focus scoring on high-risk workflows, RAG answers, tool calls, or outputs with missing citations. Scores appear directly in logs, so engineers can move from a failed answer to the prompt, model call, retrieval step, tool output, latency, token usage, and cost data behind the failure.

Flexible scorers for different hallucination patterns

Braintrust supports the scorer mix hallucination detection usually requires. Autoevals cover common patterns like factuality, while LLM-as-a-judge rubrics handle task-specific criteria. For RAG workloads, embedding-based checks score groundedness, and custom code scorers can enforce citation rules, output structure, or domain-specific constraints. Loop helps teams describe failure modes in natural language, generate SQL filters

Braintrust
reviewed examples.

Human review and regression coverage

BUILD WITH BRAINTRUST

Put these patterns to work in production.

[Docs](#) [Customers](#) [Blog](#) [Pricing](#) [Contact sales](#) [Sign in](#) [Sign up](#)

Start building

Human review remains within the same evaluation system when automated scoring requires validation. Reviewers can label traces, add corrections, assign rows for review, compare outputs, and curate production logs into **evaluation datasets**. Reviewed failures can become regression cases, which means a hallucinated production answer becomes part of the next release gate.

Best for: Production AI teams that need hallucination detection connected to evaluation, trace-level monitoring, prompt iteration, CI quality gates, and human review.

Pros

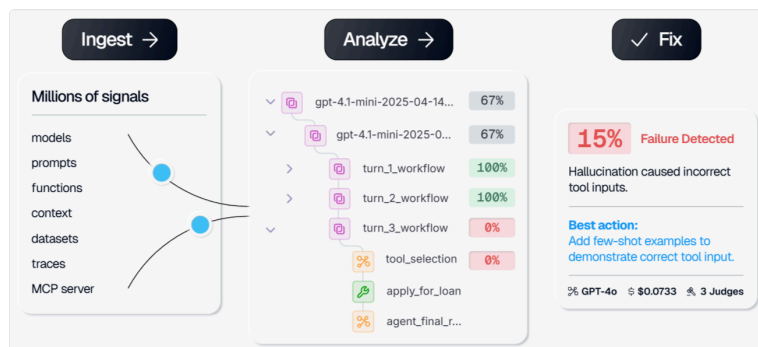
- **Unified scoring:** Runs the same hallucination scorers in pre-deployment evals and production traces.
- **CI quality gates:** Bring hallucination evals into pull requests before changes are merged.
- **Trace-level debugging:** Shows the prompt, retrieval step, tool call, model output, score, latency, tokens, and cost behind each failure.
- **Trace-to-dataset workflow:** Turns hallucinated production outputs into reusable regression cases.
- **Flexible scorers:** Supports Autoevals, LLM-as-a-judge rubrics, groundedness checks, and custom code scorers.
- **Loop support:** Helps generate datasets, scorers, filters, and prompt improvements from natural language.
- **Built-in human review:** Lets reviewers label traces, validate scorers, and curate edge cases in the same workflow.
- **Auto-instrumented tracing:** Captures supported provider calls without requiring engineers to manually wrap every model request.
- **Multi-provider support:** Works across major **model providers** and **frameworks**.
- **Accessible free tier:** Includes unlimited users and core evaluation workflows.

Cons

- No built-in fine-tuned classifier.
- No inline runtime blocking.

Pricing: The Starter plan is free and includes 1M trace spans, 10K scores, and unlimited users. Paid plans start at \$249/month, with custom enterprise pricing available. [See pricing details.](#)

2. Galileo



Galileo focuses on AI evaluation, observability, and runtime protection for GenAI applications. Its hallucination detection workflow centers on Luna-2 evaluators, prebuilt metrics such as Correctness, Context Adherence, and Chunk Attribution, and guardrail workflows for teams that need selected outputs checked before delivery. Galileo fits teams that need low-latency scoring on production traffic, while teams that need deeper scorer customization or release-control workflows may need additional evaluation infrastructure.

Best for: Teams that need low-latency hallucination scoring and runtime guardrails on high-risk LLM outputs.

Pros

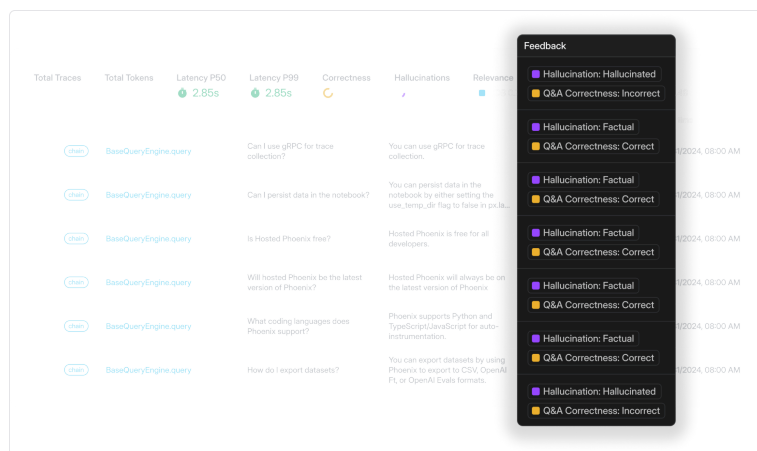
- Luna-2 supports low-latency scoring for evaluation and guardrail workflows.
- Runtime guardrails can block or route selected outputs before delivery.
- Prebuilt metrics cover correctness, context adherence, and chunk attribution.
- RAG evaluation workflows support groundedness and attribution checks.

Cons

- Detection quality depends on Galileo-maintained evaluator models.
- Custom scoring outside Galileo's metric set is more constrained.
- Real-time guardrails require Enterprise access.
- Inline blocking can add latency and over-trigger on open-ended tasks.

Pricing: Free tier with 5,000 traces/month. Paid plan starts at \$100/month. Custom enterprise pricing.

3. Arize Phoenix



Total Traces	Total Tokens	Latency P50	Latency P99	Correctness	Hallucinations	Relevance
		2.85s	2.85s			
chat	BaseQueryEngine.query	Can I use gRPC for trace collection?	You can use gRPC for trace collection.			
chat	BaseQueryEngine.query	Can I persist data in the notebook?	You can persist data in the notebook by either setting the use_temp_dir flag to false in p...			
chat	BaseQueryEngine.query	Is Hosted Phoenix free?	Hosted Phoenix is free for all developers.			
chat	BaseQueryEngine.query	Will hosted Phoenix be the latest version of Phoenix?	Hosted Phoenix will always be on the latest version of Phoenix.			
chat	BaseQueryEngine.query	What coding languages does Phoenix support?	Phoenix supports Python and TypeScript/JavaScript for auto-instrumentation.			
chat	BaseQueryEngine.query	How do I export datasets?	You can export datasets by using Phoenix to export to CSV, OpenAI FL, or OpenAI Evala formats.			

Hallucination	Q&A Correctness	Timestamp
Hallucination: Hallucinated	Q&A Correctness: Incorrect	
Hallucination: Factual	Q&A Correctness: Correct	1/2024, 08:00 AM
Hallucination: Factual	Q&A Correctness: Correct	1/2024, 08:00 AM
Hallucination: Factual	Q&A Correctness: Correct	1/2024, 08:00 AM
Hallucination: Factual	Q&A Correctness: Correct	1/2024, 08:00 AM
Hallucination: Factual	Q&A Correctness: Correct	1/2024, 08:00 AM
Hallucination: Hallucinated	Q&A Correctness: Incorrect	

Arize Phoenix gives teams a self-hostable observability and evaluation layer for LLM applications. It fits organizations that need OpenTelemetry-based tracing, infrastructure control, and RAG evaluation templates for context relevance, groundedness, and answer relevance. Phoenix is strongest when self-hosting and trace inspection are the main requirements; structured release gating and cross-functional review generally require more configuration or a move to Arize AX.

Best for: Teams that need self-hosted hallucination evaluation with LLM observability.

Pros

- Phoenix can run on team-managed infrastructure.
- RAG templates cover groundedness, context relevance, and answer relevance.
- OpenTelemetry support fits existing observability stacks.
- Arize AX adds hosted monitoring, online evals, and enterprise controls.

Cons

- Structured eval programs require more configuration.
- Phoenix centers on tracing and troubleshooting, not release enforcement.
- Managed review workflows depend on whether Arize AX or the team setup is used.
- Phoenix is not designed as an inline runtime guardrail layer.

Pricing: Free for open-source self-hosting. Managed cloud at \$50/month. Custom enterprise pricing.

4. Patronus AI

Patronus AI provides evaluator models, evaluation APIs, and domain-specific testing assets for LLM applications. Its hallucination coverage includes Lynx and other hallucination evaluators that check whether an output is grounded in the retrieved context. Patronus fits regulated or domain-specific workflows where evaluator accuracy and benchmark coverage are central, while teams usually still need separate systems for tracing, production monitoring, release gating, or broader observability.

Best for: Teams in regulated domains that need hallucination evaluators or domain-specific benchmarks.

Pros

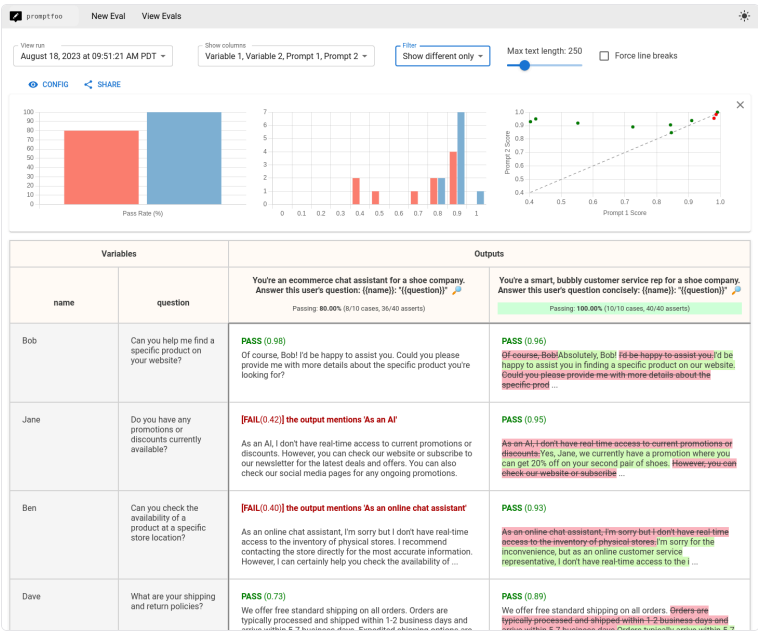
- Lynx checks whether RAG outputs are grounded in the provided context.
- FinanceBench and CopyrightCatcher support domain-specific evaluation.
- API and SDK access fit existing testing workflows.
- Custom evaluators support specialized scoring criteria.

Cons

- Teams still need tracing, monitoring, and release workflows around Patronus.
- Patronus covers evaluator workflows more than full production observability.
- Review workflows depend on how teams integrate the evaluator layer.
- Larger deployments require a custom sales discussion.

Pricing: Free Lynx model available as an open-source download. Usage-based API pricing, \$10-20 per 1,000 API calls, depending on evaluator size for hosted Patronus evaluators. Custom enterprise pricing.

5. Promptfoo



Promptfoo gives engineering teams an open-source way to run LLM evaluations, red-team tests, and vulnerability scans from configuration files and CI workflows. It fits teams that want hallucination, factuality, contradiction, and security checks close to the codebase without adopting a hosted evaluation product. Promptfoo works best for pre-deployment testing and CI; production monitoring, trace-level debugging, human review, and runtime guardrails require additional tooling or Enterprise configuration.

Best for: Engineering teams that want open-source hallucination evals in CI without a hosted account.

Pros

- Promptfoo runs locally or in self-hosted environments.

- Factuality assertions support checks against reference answers.
- Hallucination red teaming tests whether models fabricate answers when uncertain.
- YAML-based tests are well-suited to GitHub Actions, GitLab CI, and similar workflows.

Cons

- Community usage does not provide trace-level production monitoring.
- Human review and cross-functional workflows require separate processes.
- Promptfoo is not an inline runtime guardrail layer.
- CLI and configuration-file workflows are less accessible for non-engineering reviewers.

Pricing: Free tier with unlimited open-source use and 10k red-team probes per month. Custom enterprise pricing.

Quick comparison: best hallucination detection tools (2026)

Capability	Braintrust	Galileo	Arize Phoenix	Patronus AI	Promptfoo
Pre-deployment hallucination eval	Runs evals in experiments, playgrounds, and CI	Supports offline evals with Galileo metrics	Supports datasets, experiments, and eval templates	Runs evaluator tests through API and SDK	Runs YAML based evals before release
Production trace scoring	Scores production traces continuously	Scores production traffic through observability workflows	Available through Phoenix and AX monitoring	Requires API or SDK integration around traces	Community is not built for live trace scoring
Runtime hallucination blocking	No inline blocking layer	Enterprise guardrails can block or route outputs	Not focused on inline blocking	Possible via API, but not a sub-200ms guardrail layer	Not designed for runtime blocking
LLM-as-a-judge support	Native LLM judge and Autoevals workflows	Available through evaluation metrics	Supports LLM-as-judge evaluations	Judge evaluators support rubric-based scoring	Supports model-graded assertions
Custom scorers	Code, LLM judge, and Autoeval scorers	Custom logic is more metric-led	Code and LLM evals supported	Custom evaluator options available	Custom assertions and plugin supported
RAG groundedness scoring	Supports groundedness scorers	Includes context adherence and attribution metrics	RAG triad covers context, groundedness, and answer relevance	Lynx checks hallucination against provided context	Requires configuration with source context
CI/CD eval gates	GitHub Action runs evals on pull requests	Requires workflow configuration	Requires team-configured CI workflow	Requires custom API/SDK workflow	CI-friendly by design
Side-by-side regression	Compares experiments	Supports comparison	Available through	Available through	Compares eval runs

Capability	Braintrust	Galileo	Arize Phoenix	Patronus AI	Promptfoo
comparison	and score deltas	inside eval workflows	experiments, with more setup	evaluation comparisons	from configurat
Trace-level debugging	Shows prompts, spans, scores, tokens, cost, and errors	Provides trace and agent workflow views	Provides trace and span inspection	Depends on Patronus tracing setup	Limited to eval and te outputs
Production logs to versioned datasets	Converts production logs to versioned datasets	Requires a configured review workflow	Possible through dataset workflows	Requires API/SDK workflow setup	No native production trace-to-dataset workflow
Human review workflow	Built-in review, labels, corrections, and annotation scores	Supports human review workflows	Human annotation available in AX workflows	Annotation support exists, but the workflow depends on the setup	Community relies on separate review processes
OpenTelemetry support	Supports OTEL-based tracing	Supports OTEL integrations	OpenTelemetry is listed in observability features	Not positioned as OTEL-native	Not positioned as OTEL-native
Auto-instrumented LLM tracing	Auto-instrumentation reduces manual setup	SDK and integrations support tracing	Instrumentation available through OTEL and integrations	SDK-based evaluator integration	Eval framework not an auto-tracing layer
Self-hosting	Enterprise self-hosting available	Enterprise deployment options available	Phoenix is free and open source	Lynx/evaluator components available, broader platform is managed	Community runs locally or self-hosted
Free plan or OSS access	Free plan with 1M trace spans, 10K scorers, and unlimited users	Free plan includes 5,000 traces	Free OSS	Free Lynx OSS	Free OSS

Start free with Braintrust to turn hallucinated production outputs into eval cases that protect future releases.

What to look for in a hallucination detection tool

Use the criteria below to compare how each hallucination detection tool fits your release process, production traffic, and review workflow.

Lifecycle coverage: Decide whether your team needs pre-deployment evaluation, production monitoring, runtime blocking, or a combination of these. A scorer that can move from eval runs to production traces reduces duplicate implementation. A tool that covers only one stage can still meet narrow needs, but engineering teams may need another system when release testing and live monitoring diverge.

Scoring flexibility: Hallucination detection rarely relies on a single method. Production teams often combine LLM-as-a-judge rubrics, groundedness checks, citation validation, deterministic rules, and fine-tuned detectors. Choose a tool that supports multiple

scorer types so evaluation criteria can evolve as prompts, retrieval systems, and model choices change.

Integration overhead: Setup effort determines how quickly engineers can start scoring outputs. Auto-instrumentation, SDK coverage, OpenTelemetry support, and CI integrations reduce the work required to connect tracing, scoring, and release checks. Manual setup can still work, but every extra integration step slows adoption across services.

RAG support: Retrieval-augmented applications need scorers that can check whether an answer is supported by retrieved context. Groundedness, faithfulness, citation validity, context relevance, and answer relevance are the core checks to look for. Built-in templates save engineering time, especially when hallucinations stem from stale documents, irrelevant chunks, or unsupported synthesis.

Multi-provider support: Teams that compare or route across OpenAI, Anthropic, Google, Azure, Bedrock, Vertex, and open-source models need hallucination detection that works across providers. Detection tied too closely to one model provider can make model swaps harder and weaken comparisons across experiments.

Free tier or open-source access: Early evaluation work usually starts before procurement. A free tier or open-source option lets engineers test scorers on real prompts, retrieved context, and production-like traces before committing to a paid plan. Low-friction access also helps product, QA, and subject-matter experts join the review process earlier.

Why Braintrust is the best choice for hallucination detection

Braintrust is the strongest choice because hallucination detection is integrated into the release process for prompt changes, model swaps, retrieval updates, and agent workflows. Teams can connect factuality checks to production traces, evaluation datasets, human review, and release gates, so hallucinated outputs from live traffic strengthen future regression coverage. For AI teams that need hallucination detection to reduce release risk, Braintrust provides the most complete operating model across evaluation, monitoring, review, and production control.

Teams including [Stripe](#), [Vercel](#), [Zapier](#), [Airtable](#), and [Instacart](#) use Braintrust for production AI observability and evaluation. [Start free with Braintrust to catch hallucinated responses before users see them →](#)

FAQs: best hallucination detection tools for LLM applications (2026)

What's the difference between hallucination detection and hallucination guardrails?

Hallucination detection scores model outputs for factuality, groundedness, contradiction, or citation support so engineering teams can find failures, improve scorers, and strengthen eval datasets. Hallucination guardrails inspect outputs before delivery and can block, rewrite, or route high-risk responses for review. Most teams start with detection across evals and production monitoring, then add guardrails only where a hallucinated answer could cause user-visible harm or pose a compliance risk.

Is LLM-as-a-judge reliable enough for production hallucination detection?

LLM-as-a-judge can be reliable in production when the rubric is grounded in concrete examples and validated against human labels on a recurring cadence. Two failure modes show up most often: rubrics that ask the judge to make subjective calls without sufficient anchoring (e.g., "is the answer faithful?" with no working definition of "faithful"), and judge drift that goes unnoticed when prompts, retrieval, or upstream models shift. Treating the judge as a versioned dependency that gets revalidated whenever upstream pieces change is the practical fix. **Braintrust** supports this by versioning judge rubrics, replaying them against labeled datasets, and surfacing score drift between experiments.

How does hallucination detection work for RAG applications?

RAG hallucination detection checks whether the answer is supported by the retrieved context. The core checks usually include context relevance, groundedness, answer relevance, citation validity, and contradiction against source material. **Braintrust** can evaluate those failures through LLM-as-a-judge rubrics, groundedness scorers, custom code scorers, and production trace scoring, which helps teams connect a bad answer to the retrieved chunks, prompt version, model call, or tool output behind the failure.

Can I use multiple hallucination detection methods at once?

Stacking methods only work if each scorer has a clear job and the team has a plan for disagreement. A common starter stack uses cheap deterministic checks (citation presence, format adherence) as the first filter, an LLM-as-a-judge for broader factuality, a groundedness scorer for RAG outputs, and human review reserved for high-risk or low-agreement traces. The harder question is what to do when scorers disagree, since two judges can return contradictory verdicts on the same output. Treating scorer agreement itself as a signal works well: high-agreement passes ship, high-agreement failures get blocked or flagged, and mixed-agreement traces route to human review. **Braintrust** supports this by running multiple scorers per trace and exposing their raw scores, enabling teams to build routing logic based on the aggregate signal.

How accurate are automated hallucination detection tools compared to human review?

Accuracy depends on the task, dataset, scoring method, and definition of hallucination, so broad percentage claims can be misleading without a specific benchmark and methodology. Automated scorers are useful because they scale across test sets and production traffic, while human reviewers are still needed to validate edge cases, resolve ambiguous examples, and improve scorer quality. **Braintrust** supports both modes by running automated online scoring on production traces and capturing structured human judgment to serve as ground truth, validate scorers, and curate datasets.

Which is the best hallucination detection tool?

Braintrust is the best overall hallucination detection tool for production AI teams that need integrated fact-checking, evaluation, production monitoring, human review, and release control. Galileo is a better fit when runtime blocking is the main requirement, Arize Phoenix fits teams prioritizing OSS self-hosting, Patronus AI fits domain-specific evaluator needs, and Promptfoo fits CI-native evals for engineering-led teams. **Braintrust** is the strongest default when hallucination detection needs to improve release decisions and prevent recurring factual failures from reaching production.

Does Braintrust have a free tier for hallucination detection?

Braintrust's free Starter plan with 1M trace spans and 10K scorers covers most early evaluation work: building a hallucination test set, running it across prompt and model variants, scoring outputs with LLM-as-a-judge or custom code scorers, and reviewing failures in traces. Teams typically use the free tier to validate that automated scoring lines up with their definition of hallucination before scaling to production traffic.

SHARE [X](#) [in](#) [🔗](#)

[< PREVIOUS](#)

[NEXT >](#)

Best RAG Evaluation Tools in
2026, Compared

Best RAG observability tools (2026):
monitor retrieval and generation
in production

Trace everything

Enter your email [Sign up](#)



RESOURCES

Documentation
Integrations
Eval foundations
Encyclopedia
Cookbook
Changelog
For PMs
Articles

COMPANY

Pricing
Blog
Careers
Contact us
Manifesto
Privacy Policy
Trust center
Legal

COMMUNITY

GitHub
Discord
Newsletter
X
YouTube
LinkedIn

